

>

THE UNIVERSAL FAANG DP THINKING PARADIGM — SB → RBR → MTS

SB — State & Brainstorm

S (State): What variables uniquely identify `dp[i]` or `dp[i][j]`?

B (Brainstorm): "If I'm at this state, what choices do I have?" (Take/Skip, Climb 1/2 steps, Rob/Skip).

RBR — Recurrence, Base, Result

R (Recurrence): `dp[i] = max(take, skip)`.

B (Base Case): Recursion stop point (`dp[0] = 0`).

R (Result): Target location (`dp[N]`, `dp[M][N]`).

MTS — Memo, Tabulate, Space

M (Memoization): Top-down recursion cache.

T (Tabulation): Bottom-up iterative table.

S (Space Opt): Compress 1D/2D table into `prev1, prev2` variables!

Golden Rule: Brute Force → Recursion → SB → RBR → Memoization → Tabulation → Space Optimization O(1).

THE 2 MANDATORY DP CONDITIONS

- Overlapping Subproblems:** Exact same state `fib(3)` computed multiple times in recursion tree.
- Optimal Substructure:** Optimal solution of size N built from optimal solutions of smaller subproblems!

CLUES THAT SCREAM DYNAMIC PROGRAMMING

- "Max / Min ways to reach target" → 1D / Knapsack DP
- "Longest common subsequence / Edit distance" → 2D String DP
- "Can partition array into 2 equal sum subsets" → 0/1 Knapsack DP

>

1 TOP-DOWN MEMOIZATION TEMPLATE

```
public int solveTopDown(int n) {
    Integer[] memo = new Integer[n + 1];
    return helper(n, memo);
}

private int helper(int n, Integer[] memo) {
    if (n ≤ 1) return n;
    if (memo[n] ≠ null) return memo[n]; // Memoization Cache Hit!
    memo[n] = helper(n - 1, memo) + helper(n - 2, memo);
    return memo[n];
}
```

2 BOTTOM-UP O(1) SPACE COMPRESSION

```
// Fibonacci / House Robber O(1) Space Compression:
public int rob(int[] nums) {
    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int curr = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

>

PATTERN 1

CLIMBING STAIRS (1D Fibonacci Transition)

e.g. Climbing Stairs (LC 70), Min Cost Climbing Stairs (LC 746)

RECURRENCE FORMULA

`dp[i] = dp[i-1] + dp[i-2]`. Space compressed to $O(1)$ variables `prev1` and `prev2`.

TEMPLATE — LC 70

```
public int climbStairs(int n) {
    if (n ≤ 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i ≤ n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

PATTERN 2

HOUSE ROBBER I (Non-Adjacent Element Selection)

e.g. House Robber I (LC 198)

CHOICE EQUATION

`dp[i] = Math.max(dp[i-1], dp[i-2] + nums[i])`. Space compressed to $O(1)$ variables.

TEMPLATE — LC 198

```
public int rob(int[] nums) {
    int prev2 = 0, prev1 = 0;
    for (int num : nums) {
        int curr = Math.max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

>

PATTERN 3

HOUSE ROBBER II (Circular Array Split)

e.g. House Robber II (LC 213)

CIRCULAR ROBBER TRICK

For circular houses, max robbery $\$ = \max(\text{rob}(0..N-2), \text{rob}(1..N-1))\$$

TEMPLATE — LC 213

```
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    return Math.max(robRange(nums, 0, nums.length - 2), robRange(nums, 1, nums.length - 1));
}
private int robRange(int[] nums, int start, int end) {
    int prev2 = 0, prev1 = 0;
    for (int i = start; i ≤ end; i++) {
        int curr = Math.max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

PATTERN 4

LONGEST PALINDROMIC SUBSTRING (Expand Around Center)

e.g. Longest Palindromic Substring (LC 5)

CENTER EXPANSION STRATEGY

Expand around $2N-1$ centers (odd center `i`, `i` and even center `i, i+1`) in $O(N^2)$ time and $O(1)$ space!

TEMPLATE — LC 5

```
public String longestPalindrome(String s) {
    int start = 0, maxLen = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expand(s, i, i); // Odd center
        int len2 = expand(s, i, i + 1); // Even center
        int len = Math.max(len1, len2);
        if (len > maxLen) { maxLen = len; start = i - (len - 1) / 2; }
    }
    return s.substring(start, start + maxLen);
}
private int expand(String s, int l, int r) {
    while (l >= 0 & r < s.length() & s.charAt(l) == s.charAt(r)) {
        l--; r++;
    }
    return r - l - 1;
}
```

```
while (l ≥ 0 && r
```

>

PATTERN 5 COIN CHANGE I (Fewest Coins to Make Amount) e.g. Coin Change (LC 322)

UNBOUNDED FORWARD LOOP

Infinite reuse of coins! Loop `a = coin..amount` forward: `dp[a] = Math.min(dp[a], 1 + dp[a - coin])`.

TEMPLATE — LC 322

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int a = 1; a ≤ amount; a++) {
        for (int coin : coins) {
            if (a ≥ coin) dp[a] = Math.min(dp[a], 1 + dp[a - coin]);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

PATTERN 6 COIN CHANGE II (Total Combination Ways) e.g. Coin Change II (LC 518)

OUTER LOOP = COINS (PREVENTS DUPLICATES)

Outer loop over `coins`, inner loop over `amount` ensures combinations (not permutations) are counted!

TEMPLATE — LC 518

```
public int change(int amount, int[] coins) {
    int[] dp = new int[amount + 1];
    dp[0] = 1;
    for (int coin : coins) {
        for (int a = coin; a ≤ amount; a++) {
            dp[a] += dp[a - coin];
        }
    }
    return dp[amount];
}
```

>

PATTERN 7 PARTITION EQUAL SUBSET SUM (0/1 Knapsack Reverse Loop) e.g. Partition Equal Subset Sum (LC 416)

REVERSE LOOP SINGLE USE TRICK

For single-use elements, inner loop MUST run backwards from `target` down to `num` to prevent using same element twice in 1D array!

TEMPLATE — LC 416

```
public boolean canPartition(int[] nums) {
    int sum = 0; for (int n : nums) sum += n;
    if (sum % 2 ≠ 0) return false;
    int target = sum / 2;
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;
    for (int num : nums) {
        for (int j = target; j ≥ num; j--) { // Reverse loop for 0/1!
            dp[j] = dp[j] || dp[j - num];
        }
    }
    return dp[target];
}
```

>

PATTERN
8

LONGEST INCREASING SUBSEQUENCE (Binary Search Tails Array)

e.g. Longest Increasing Subsequence (LC 300)

PATIENCE SORTING $O(N \log N)$

Maintain array `tails[]` of smallest tail of all increasing subsequences of length $i+1$. Use binary search (`Arrays.binarySearch`) to replace or append!

TEMPLATE — LC 300

```
public int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];
    int len = 0;
    for (int num : nums) {
        int i = Arrays.binarySearch(tails, 0, len, num);
        if (i < 0) i = -(i + 1);
        tails[i] = num;
        if (i == len) len++;
    }
    return len;
}
```

>

PATTERN 9

LONGEST COMMON SUBSEQUENCE (2D Matrix Match)

e.g. Longest Common Subsequence (LC 1143)

MATCHING VS NON-MATCHING TRANSITION

If `text1[i-1] == text2[j-1]`: `dp[i][j] = 1 + dp[i-1][j-1]`. Else `dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1])`.

TEMPLATE — LC 1143

```
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}
```

PATTERN 10

EDIT DISTANCE (Insert, Delete, Replace Operations)

e.g. Edit Distance (LC 72)

3 EDIT TRANSITIONS

`dp[i][j] = 1 + Min(Insert dp[i][j-1], Delete dp[i-1][j], Replace dp[i-1][j-1])`.

TEMPLATE — LC 72

```
public int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1];
            else dp[i][j] = 1 + Math.min(dp[i - 1][j], Math.min(dp[i][j - 1], dp[i - 1][j]));
        }
    }
    return dp[m][n];
}
```

>

DYNAMIC PROGRAMMING — WHICH PATTERN TO USE?

Start: What is the DP problem type?

1D Steps / Non-adjacent items

→

1D DP `'prev'` / `O(1)` and Space `'prev2'`

Unbounded item reuse (Coin Change)

→

Unbounded Knapsack

Single-use items (Subset Sum)

→

0/1 Knapsack

2-String matching (LCS / Edit)

→

2D `'dp[m+1][n+1]'` DP grid

Reverse loop `'j = target..n'`

Subsequence ordering (LIS)

→

Patience Sorting LIS

Binary search `'tails'` array

TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Fewest coins to amount"	Unbounded Knapsack	Forward loop <code>'1 + dp[a - coin]'</code>
"Partition equal sum"	0/1 Knapsack	Reverse loop <code>'dp[j] dp[j - num]'</code>
"Longest increasing subsequence"	LIS Patience Sort	Binary search replacement in <code>'tails'</code>
"Edit distance", "Min operations"	2D String DP	<code>'1 + Min(Insert, Delete, Replace)'</code>

INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why reverse loop	A forward loop re-uses the SAME item multiple times (unbounded). A reverse

>

 DRY RUN — Coin Change ([1, 2, 5], Amount=11)

Amount	dp[a] Calculation	Best Coin Used	dp[a] Value
0	Base case	-	0
1	\$1 + dp[0] = 1\$	Coin 1	1
2	\$\min(1+dp[1], 1+dp[0]) = 1\$	Coin 2	1
5	\$\min(1+dp[4], 1+dp[0]) = 1\$	Coin 5	1
11	\$1 + dp[6] = 1 + 2 = 3\$ (Coins 5+5+1)	Coin 5	3 

 MATHEMATICAL PROOF — O(N LOG N) LIS PATIENCE SORT

Claim: `tails[i]` stores the smallest tail of all increasing subsequences of length i+1.

Proof: `tails[]` is strictly increasing! For each number `x`, binary search finds the first element `tails[i] >= x`. Replacing `tails[i]` with `x` maintains the invariant with a smaller tail, creating more capacity for future elements $\rightarrow$ **O(N log N) Time!**

 MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write 1D Climbing Stairs in O(1) space

☐ Code House Robber II circular range split

☐ Implement Coin Change I unbounded loop

☐ Code Coin Change II combination loop

☐ Write 0/1 Knapsack reverse loop

☐ Code LIS in O(N log N) with binary search

☐ Write Longest Common Subsequence (LC 1143)

☐ Code Edit Distance (LC 72) in 2D grid

 GOLDEN RULES — NEVER FORGET

Rule 1 — Fill Array with Safe Infinity
In min-cost DP (e.g. Coin Change), initialize `dp[]` with `amount + 1` (NOT `Integer.MAX\_VALUE` to avoid overflow during `1 + dp[a - coin]`)!

Rule 2 — Reverse Inner Loop for 0/1 Knapsack
When elements can only be used ONCE, inner loop MUST run backwards (`for (int j = target; j >= num; j--)`)!